

SYSTEM DESIGN INTERVIEW · CHAPTER 15

# 設計 Google Drive

雲端檔案儲存與同步系統：如何用最少的頻寬，讓數千萬台裝置看到同一份檔案

團隊技術分享 · 20 頁完整導讀



# 四個步驟，一條主線

1

## 釐清需求與範圍

功能／非功能需求、規模估算，找出真正的瓶頸

2

## 高階設計（演進式）

從單機一路撞牆到分散式架構，八個元件各就各位

3

## 深入設計（重點）

分塊儲存、增量同步、一致性、上傳下載流程、通知與容錯

4

## 總結與取捨

替代方案的優缺點，以及面試官會追問什麼

今天最重要的一句話

資料面  
與  
控制面  
分離

## STEP 1

# 需求釐清：先把題目縮小

### 功能需求（本章範圍）

- 新增檔案（拖放上傳）
- 下載檔案
- 跨多裝置自動同步
- 查看檔案修訂版本
- 與他人分享檔案
- 檔案被編輯／刪除／分享時發出通知

### 非功能需求

- 可靠性 — 資料遺失絕不可接受
- 同步速度快 — 慢了使用者就跑了
- 省頻寬 — 尤其是行動網路
- 可擴展性 — 撐得住大量流量
- 高可用 — 部分節點掛掉仍可服務

#### 平台

Web + Mobile 都要支援

#### 加密

儲存中（at rest）必須加密

#### 檔案上限

單檔 10 GB，型別不限

## STEP 1

# 規模估算：數字要會推，不是背

**50M**

註冊用戶

題目給定

**10M**

日活躍 DAU

題目給定

**500 PB**

總配置空間

$50M \times 10 \text{ GB}$

**480**

尖峰上傳 QPS

$10M \times 2 \div 86400 \approx 240, \times 2$

### 兩個容易被追問的點

- 500 PB 是「配額總和」不是「實際用量」——多數人用不滿 10 GB，實務上一定會超賣（oversubscription）
- 假設每人每天上傳 2 個檔案、平均 500 KB、讀寫比 1:1

# 真正的瓶頸在哪裡？

# 480 QPS

一台像樣的機器就吃得下。

這個系統的瓶頸不是「請求數」。

## 瓶頸是 → 頻寬 與 儲存量

所以本章後面「每一個」優化，打的都是這兩個瓶頸：

- 分塊 + 增量同步 → 頻寬
- 壓縮 → 頻寬
- 區塊去重（dedupe） → 儲存
- 版本數上限 → 儲存
- 冷資料移到 Glacier → 儲存成本

# 演進式推導：撞一次牆，修一次



重點不是結論，是「每個元件都被某個具體問題逼出來」——面試時用這個敘事，比直接畫大圖更有說服力。

# 三支 API 就夠了

## 1 上傳檔案

```
POST /files/upload?  
uploadType=resumable
```

- 簡單上傳：小檔直接送
- 可恢復上傳：大檔或網路不穩
- 參數：uploadType、data

## 2 下載檔案

```
GET /files/download
```

- 參數：path
- 例：{ "path":  
"/recipes/soup/best.txt" }
- 回傳檔案內容

## 3 取得修訂版本

```
GET /files/list_revisions
```

- 參數：path、limit
- limit = 最多回傳幾個版本
- 支撐「版本歷史」功能

安全基線：所有 API 都需要使用者身分驗證，一律走 HTTPS，用 SSL/TLS 保護客戶端與後端之間的傳輸。

## STEP 2

# 高階架構：八個元件各司其職

### Block Servers

分塊、壓縮、加密，再上傳雲儲存

### Cloud Storage (S3)

存放實際的資料區塊

### Cold Storage

存放長期不被存取的冷資料

### Load Balancer

把請求平均分到 API 伺服器

### API Servers

認證、用戶資料、更新中繼資料（上傳流程除外）

### Metadata DB

用戶、檔案、區塊、版本等中繼資料

### Metadata Cache

快取熱門中繼資料以加速讀取

### Notification Service

Pub/Sub，事件發生時通知客戶端

### Offline Backup Queue

客戶端離線時暫存變更，上線後補同步

最該記住的一句：檔案本體在雲端，Metadata DB 裡只有中繼資料。



# 看懂這張圖的鑰匙：兩條路徑

## 資料面 Data Plane

客戶端 → Block Servers → S3

- 搬的是「檔案位元組」，量大、慢
- 在這裡做分塊、壓縮、加密
- 所有省頻寬的優化都發生在這條路
- 唯一不歸 API Servers 管的流程

## 控制面 Control Plane

客戶端 → API Servers → Metadata DB

- 搬的是「誰有什麼、哪個版本」，量小、要快
- 認證、權限、版本、狀態機都在這裡
- 所有一致性的要求都壓在這條路
- 通知服務也屬於這一側

上傳時兩條路「並行」跑；下載時先走控制面拿 metadata，再走資料面拿區塊 —— 順序不能反。

### STEP 3

## Block Servers：全章的核心



**4 MB**

單個區塊大小上限（參考 Dropbox 的做法）

**Hash**

每個區塊有唯一雜湊值，存在 Metadata DB —— 這也是後面「去重」的基礎

**Order**

每個區塊在 S3 是獨立物件；重建檔案要按 `block_order` 正確順序串接

為什麼加密放在這裡，而不是客戶端？因為客戶端容易被駭或被操縱 —— 最後一頁會再回到這個取捨。

### STEP 3

## 增量同步：只傳改動過的區塊

本機的 10 個區塊



只上傳  
這 2 塊 →

Cloud Storage

Block 2

Block 5

### 增量同步 Delta Sync

改檔案時只同步被修改的區塊，不是整份檔案。底層是 rsync 這類同步演算法

### 壓縮 Compression

依檔案型別選演算法：文字用 gzip / bzip2，圖片與影片需要另一套壓縮法

效果：一個 5 GB 的檔案改了幾行字，實際上傳量可能只有幾 MB。

# 強一致性：為什麼選關聯式資料庫

同一個檔案，在不同客戶端顯示不同內容 —— 這是不可接受的。所以中繼資料快取與資料庫層都必須強一致。

## 要做到強一致，必須確保

- 快取副本與主伺服器的資料一致
- 資料庫寫入時讓快取失效（invalidate），確保快取與 DB 值相同

## 為什麼不是 NoSQL ？

- 關聯式 DB 原生支援 ACID（原子性、一致性、隔離性、持久性）
- NoSQL 預設不支援 ACID，要自己在同步邏輯裡用程式實作，容易出錯



面試官最愛戳的取捨：選了關聯式 DB，就是拿「水平擴展的彈性」去換「一致性」。這裡敢換，是因為第 5 頁算過 —— 尖峰只有 480 QPS。

### STEP 3

## 中繼資料表設計

#### user

使用者基本資料

user\_id · user\_name · created\_at

#### device

一個用戶可有多台裝置

device\_id · user\_id · push\_id

#### namespace

使用者的根目錄

id · owner\_id · is\_shared

#### file

最新版檔案的所有資訊

file\_name · relative\_path · latest\_version

#### file\_version

版本歷史；現有列唯讀

file\_id · version\_number · last\_modified

#### block

區塊資訊，重建檔案用

block\_id · file\_version\_id · block\_order

兩個必記的細節：**file\_version** 的現有列是「唯讀」的（**append-only**），版本歷史才可信；**block** 一定要有 **block\_order**，否則檔案重建不回來。

## STEP 3

# 上傳流程：兩條路並行跑

### A · 控制面：新增 metadata

- 1 Client 1 請求新增檔案的 metadata
- 2 存入 Metadata DB，上傳狀態設為 pending
- 3 通知 Notification Service：有新檔案要進來
- 4 Notification Service 通知 Client 2：有檔案正在上傳

### B · 資料面：上傳檔案內容

- 1 Client 1 把檔案內容傳給 Block Servers
- 2 分塊、壓縮、加密，上傳到 S3
- 3 上傳完成，S3 觸發 callback 到 API Servers
- 4 Metadata DB 狀態改為 uploaded
- 5 通知 Notification Service 狀態已變更
- 6 Notification Service 通知 Client 2：已完整上傳

**關鍵：** pending → uploaded 這個兩階段狀態機，讓其他裝置能先知道「有東西要來了」，但不會去下載一個還沒傳完的檔案。

### STEP 3

## 下載流程：先要 metadata，再抓區塊

#### 客戶端怎麼知道檔案變了？

在線 → Notification Service 主動通知，客戶端去拉最新資料

離線 → 變更存進 Offline Backup Queue，客戶端上線後補拉

- 1 Notification Service 通知客戶端：某檔案在別處被改了
- 2 Client 2 發出取得 metadata 的請求
- 3 API Server 向 Metadata DB 查詢變更的 metadata
- 4 Metadata DB 回傳給 API Server
- 5 Client 2 取得 metadata（知道需要哪些區塊）
- 6 Client 2 向 Block Servers 請求下載區塊
- 7 Block Servers 先從 S3 下載區塊
- 8 S3 把區塊回傳給 Block Servers
- 9 Client 2 下載所有新區塊，重建檔案

步驟 1-5 走控制面（淺色圓點），6-9 走資料面（深色圓點）。順序不能反——沒有 metadata，你不知道要抓哪些區塊。

### STEP 3

# 通知服務：為什麼選 Long Polling

## ✓ Long Polling（採用）

- Dropbox 採用的方案
- 客戶端與通知服務保持長輪詢連線
- 偵測到變更 → 關閉連線 → 連到 metadata server 下載變更 → 立刻再發新請求

## ✗ WebSocket（不採用）

- 在客戶端與伺服器間提供持久連線
- 溝通是雙向的
- 適合聊天這類即時雙向通訊的場景

### 通知是單向的

伺服器把檔案變更推給客戶端，反過來則不需要

### 沒有突發資料

雲端硬碟的通知不頻繁，不需要持久雙向通道的成本



中文版翻譯勘誤：原書寫「反之亦然」，英文原文是 **not vice versa**，應為「反之則不然」。翻反了會讓「所以選單向的 Long Polling」這個推論自相矛盾。



### STEP 3

## 同步衝突：先到先贏

策略：先被系統處理的版本獲勝，後到的收到衝突

- 1 User 1 與 User 2 同時修改同一個檔案
- 2 User 1 的版本先被系統處理 → 更新成功
- 3 User 2 發生同步衝突
- 4 系統給 User 2 兩份：他的本地副本 + 伺服器最新版

```
SystemDesign_user2_conflicted_copy_2019-05-01
```

要講清楚的範圍界線

這是「檔案層級」的衝突處理，  
不是 Google Docs 那種即時協作編輯。

真正的多人同時編輯要靠  
OT（操作轉換）或 CRDT，  
書中以 Differential Synchronization  
作為延伸閱讀帶過。

被追問時能講出這個區別，  
就是深度。

設計哲學：系統不猜使用者要什麼。偵測到衝突就保留雙方版本，把決定權交還給人。

# 節省儲存空間：三招

問題：為了支援版本歷史與可靠性，同一個檔案的多個版本被存在多個資料中心。全部照存，空間很快就被填滿。

## 01

### 區塊去重 Dedupe

- 在帳戶層面消除多餘的區塊
- 兩個區塊 hash 相同 → 就是同一個區塊，只存一份
- 最簡單也最有效的一招

## 02

### 智慧備份策略

- 設版本數上限：達到上限後，最舊的版本被新版本替換
- 只保留有價值的版本：對近期版本給更高權重
- 最佳版本數靠實驗找出來

## 03

### 冷資料移到冷儲存

- 冷資料 = 數月或數年未被存取的資料
- 移到 Amazon S3 Glacier
- 成本比一般 S3 便宜得多

**提醒：**頻繁編輯的檔案可能在短時間內被存 **1000 多次** —— 沒有版本上限，儲存成本會失控。

### STEP 3

## 故障處理：每個元件掛了怎麼辦

#### Load Balancer

備援節點接手；靠 heartbeat 互相監控，一段時間沒心跳即判定失敗

#### Block Server

其他伺服器接管未完成或掛起的作業

#### Cloud Storage

S3 bucket 跨地域多次複製，一區不可用就從別區取

#### API Server

無狀態服務，負載平衡器直接把流量導到其他台

#### Metadata Cache

多副本；掛一個節點仍可讀其他節點，並啟動新快取伺服器替換

#### Metadata DB

Master 掛 → 提升一個 slave 為新 master，再拉一台新 slave；Slave 掛 → 用其他 slave 讀，補上新機器

#### Notification Service

每台機器維持超過 100 萬個長輪詢連線（2012 Dropbox 數據）；掛掉後連線全失，重連是相對緩慢的過程

#### Offline Backup Queue

佇列多副本；佇列掛掉時，消費者重新訂閱備用佇列

面試提示：這一頁是最容易被追問的地方。回答的通用套路是「多副本 + 自動故障轉移 + 無狀態化」，再針對元件補特殊之處。

# 總結與取捨

## 替代方案：客戶端直傳雲儲存（繞過 Block Servers）

### 優點

上傳更快 —— 檔案只需傳輸一次到雲儲存，不必先經過 Block Servers 再轉一手

### 缺點

- ① 分塊、壓縮、加密邏輯要在 iOS、Android、Web 各實作一遍 —— 易錯且工程量大
- ② 客戶端容易被駭或被操縱，在客戶端實作加密邏輯並不理想

## 帶走三件事

- 1 資料面與控制面分離
- 2 強一致性換掉部分擴展彈性，因為 QPS 不高
- 3 分塊是省頻寬與省儲存的共同基礎

### 延伸討論

把在線／離線邏輯抽成獨立的 Presence Service，從通知伺服器移出來，其他服務就能輕鬆整合。 | 沒有完美解法：每家公司限制不同，重點是講得清楚你的取捨。